Sum-Check Toolkit

July 17, 2024

Distributed Lab

zkdl-camp.github.io

github.com/ZKDL-Camp

Sum-Check-based Protocol for Grand Products

Grand Product Relation

$$\mathcal{R}_{GP} = \left\{ (p \in \mathbb{F}, \mathbf{v} \in \mathbb{F}^m) \colon p = \prod_{i=0}^{m} v_i \right\}$$

Assume that m is a power of 2.

Grand Product Relation

$$\mathcal{R}_{GP} = \left\{ (p \in \mathbb{F}, \mathbf{v} \in \mathbb{F}^m) \colon p = \prod_{i=0}^{m} v_i \right\}$$

Assume that m is a power of 2.

Let $\widetilde{v}$ be an MLE of $\mathbf{v}$, by viewing $\mathbf{v}$ as a function mapping $\{0,1\}^{\log m} \to \mathbb{F}$.

Main Lemma

Lemma

A scalar p and a vector $\mathbf{v}$ satisfies the relation $\mathcal{R}_{GP}$ if and only if there exists a multilinear polynomial f in $\log m + 1$ variables such that $f(1, \ldots, 1, 0) = p$ and $\forall x \in \{0,1\}^{\log m}$ the following hold:

$$f(0, x) = v(x)$$
$$f(1, x) = f(x, 0) \cdot f(x, 1)$$

Such polynomial f has the following construction:

- $f(1, \ldots, 1) = 0$
- *For all $\ell \in [\log m]$ and $x \in \{0,1\}^{\log m - \ell}$:*

$$f(1^{\ell}, 0, x) = \prod_{y \in \{0,1\}^{\ell}} v(x, y)$$

Example

Let $m = 4$ $(\log m = 2)$, $\mathbf{v} = \{1, 2, 3, 4\}$, consequently $p = 1 \times 2 \times 3 \times 4 = 24$, then:

Example

Let $m = 4$ $(\log m = 2)$, $\mathbf{v} = \{1, 2, 3, 4\}$, consequently $p = 1 \times 2 \times 3 \times 4 = 24$, then:

$$v(x_1, x_2) = 1 + 2x_1 + x_2$$

$$v(x_1, x_2): \quad v(0,0) = 1,\ v(0,1) = 2,\ v(1,0) = 3,\ v(1,1) = 4.$$

Example

Let $m = 4$ ($\log m = 2$), $\mathbf{v} = \{1, 2, 3, 4\}$, consequently $p = 1 \times 2 \times 3 \times 4 = 24$, then:

$$v(x_1, x_2) = 1 + 2x_1 + x_2$$

$$v(x_1, x_2): \quad v(0,0) = 1,\ v(0,1) = 2,\ v(1,0) = 3,\ v(1,1) = 4.$$

Now, we define f as follows:

$$f(0,0,0) = 1,\ f(0,0,1) = 2,\ f(0,1,0) = 3,\ f(0,1,1) = 4,$$

Example

Let $m = 4$ ($\log m = 2$), $\mathbf{v} = \{1, 2, 3, 4\}$, consequently $p = 1 \times 2 \times 3 \times 4 = 24$, then:

$$v(x_1, x_2) = 1 + 2x_1 + x_2$$

$$v(x_1, x_2): \quad v(0,0) = 1,\ v(0,1) = 2,\ v(1,0) = 3,\ v(1,1) = 4.$$

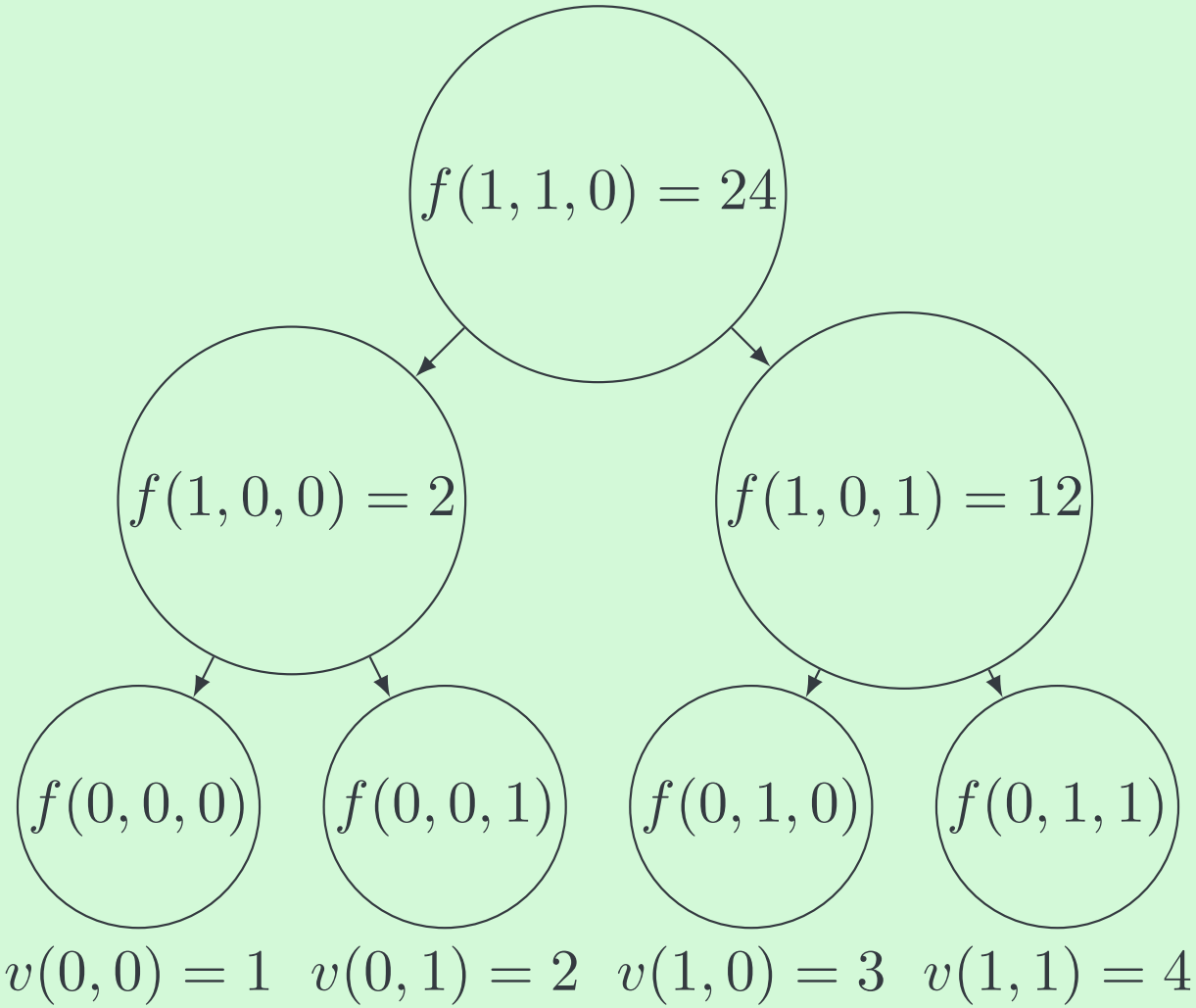
Now, we define f as follows:

$$f(0,0,0) = 1,\ f(0,0,1) = 2,\ f(0,1,0) = 3,\ f(0,1,1) = 4,$$

and:

$$\begin{aligned} f(1,0,0) &= f(0,0,0) \times f(0,0,1) = 1 \times 2 = 2, \\ f(1,0,1) &= f(0,1,0) \times f(0,1,1) = 3 \times 4 = 12, \\ f(1,1,0) &= f(1,0,0) \times f(1,0,1) = 2 \times 12 = 24 = p, \\ f(1,1,1) &= 0. \end{aligned}$$

Example



Where Is Sum-Check?

Zero-Check

$$\forall x \in \{0,1\}^{\log m} \colon f(1,x) = f(x,0) \cdot f(x,1)$$

Zero-Check

$$\forall x \in \{0,1\}^{\log m} \colon f(1,x) = f(x,0) \cdot f(x,1)$$

$$\forall x \in \{0,1\}^{\log m} \colon f(1,x) - f(x,0) \cdot f(x,1) = 0$$

Zero-Check

$$\forall x \in \{0,1\}^{\log m} \colon f(1,x) = f(x,0) \cdot f(x,1)$$

$$\forall x \in \{0,1\}^{\log m} \colon f(1,x) - f(x,0) \cdot f(x,1) = 0$$

One can use the sum-check protocol to prove the evaluation of g that is referred to a MLE of $f(1,x) - f(x,0) \cdot f(x,1)$:

$$g(t) = \sum_{x \in \{0,1\}^{\log m}} \widetilde{\mathsf{eq}}(t,x) \cdot (f(1,x) - f(x,0) \cdot f(x,1))$$

By the Schwartz–Zippel lemma, for random $\tau \in \mathbb{F}^{\log m}$, $g(\tau) = 0$ if and only if $g = 0$, except for a soundness error of $\frac{\log m}{|\mathbb{F}|}$.

Thus, to prove the existence of f and hence the grand product relationship, it suffices to prove, for some verifier selected random $\tau, \gamma \in \mathbb{F}^{\ell}$, that:

$$0 = \sum_{x \in \{0,1\}^{\log m}} \widetilde{\mathsf{eq}}(x, \tau) \cdot (f(1, x) - f(x, 0) \cdot f(x, 1)) \tag{1}$$

$$f(0, \gamma) = \widetilde{v}(\gamma) \tag{2}$$

$$f(1, \ldots, 1, 0) = p \tag{3}$$

Algorithm 1: Sum-Check-based Protocol for Grand Products

1 $\mathcal{P}$: Compute polynomials $v \in \mathbb{F}^{\log m}[x]$, $f \in \mathbb{F}^{\log m+1}[x]$ such that

$$p = \prod_{x \in \{0,1\}^{\log m}} v(x) \quad \text{and} \quad f, v \text{ satisfy } (1), (2), (3).$$

2 $\mathcal{P}$: $C_f \leftarrow \mathsf{Commit}(f)$; $C_v \leftarrow \mathsf{Commit}(v)$; send C_f, C_v to $\mathcal{V}$.

3 $\mathcal{V}$: Choose random $\tau, \gamma \in \mathbb{F}^{\log m}$ and send them to $\mathcal{P}$.

4 $\mathcal{P}$: Compute $g(x) = \widetilde{\mathrm{eq}}(x, \tau)\big(f(1,x) - f(x,0)\, f(x,1)\big)$.

5 $\mathcal{P}$ & $\mathcal{V}$: Run $\mathsf{SumCheckProtocol}(0,\ g,\ C_f)$

6 $\mathcal{V}$: $a \leftarrow \mathsf{Query}(C_f, (0, \gamma))$, $v(\gamma) \leftarrow \mathsf{Query}(C_v, \gamma)$.

7 **if** $a \neq v(\gamma)$ **then**

8 $\quad$ $\mathcal{V}$ **rejects**.

9 **end**

10 $\mathcal{V}$: $r \leftarrow \mathsf{Query}(C_f, (1, \ldots, 1, 0))$.

11 **if** $r \neq p$ **then**

12 $\quad$ $\mathcal{V}$ **rejects**.

13 **end**

Randomized Permutation Check

The goal is to verify, with high probability, whether two sequences of tuples are permutations of each other, without performing a full sort or pairwise comparison.

The goal is to verify, with high probability, whether two sequences of tuples are permutations of each other, without performing a full sort or pairwise comparison.

$$A = \{(1, 2, 3), (4, 0, 6)\}, \quad B = \{(4, 0, 6), (1, 2, 3)\}.$$

The naive approach would be to sort both sequences and then compare them.

Reed-Solomon Fingerprinting

Definition (Reed-Solomon Fingerprinting)

Let $\mathbf{a} \in \mathbb{F}^n$, then for a random $\gamma \in \mathbb{F}$, the Reed-Solomon fingerprinting of $\mathbf{a}$ is defined as:

$$h_\gamma(\mathbf{a}) = \sum_{i \in n} a_i \cdot \gamma^i.$$

Reed-Solomon Fingerprinting

Definition (Reed-Solomon Fingerprinting)

Let $\mathbf{a} \in \mathbb{F}^n$, then for a random $\gamma \in \mathbb{F}$, the Reed-Solomon fingerprinting of $\mathbf{a}$ is defined as:

$$h_\gamma(\mathbf{a}) = \sum_{i \in n} a_i \cdot \gamma^i.$$

$h_\gamma(\mathbf{a})$ uniquely identifies the sequence $\mathbf{a}$ with high probability, i.e., let $\mathbf{b} \in F^n$ and $\mathbf{a} \neq \mathbf{b}$, then, according to the Schwartz-Zippel lemma:

$$\Pr[h_\gamma(\mathbf{a}) = h_\gamma(\mathbf{b})] \leq \frac{n}{|\mathbb{F}|}.$$

Reed-Solomon Fingerprinting

Example

Consider all operations in $\mathbb{F}_7$, and set $n = 3$, $\gamma = 3$. Let

$$\mathbf{a} = (1, 2, 3), \quad \mathbf{b} = (4, 0, 6).$$

Then

$$h_\gamma(1, 2, 3) = 1 \cdot 3^0 + 2 \cdot 3^1 + 3 \cdot 3^2 = 1 + 6 + 6 = 13 \equiv 6,$$
$$h_\gamma(4, 0, 6) = 4 \cdot 1 + 0 \cdot 3 + 6 \cdot 2 = 4 + 0 + 12 = 16 \equiv 2.$$

Randomized Permutation Check

Definition (Randomized Permutation Check)

Let A and B be two multisets of tuples in $\mathbb{F}^n$. Define

$$\mathcal{H}_{\tau,\gamma}(X) = \prod_{x \in X} \big(h_\gamma(x) - \tau\big).$$

Randomized Permutation Check

Definition (Randomized Permutation Check)

Let A and B be two multisets of tuples in $\mathbb{F}^n$. Define

$$\mathcal{H}_{\tau,\gamma}(X) = \prod_{x \in X} \big(h_\gamma(x) - \tau\big).$$

Then comparing $\mathcal{H}_{\tau,\gamma}(A)$ and $\mathcal{H}_{\tau,\gamma}(B)$ yields a randomized test for whether A and B are permutations of one another.

Randomized Permutation Check

Definition (Randomized Permutation Check)

Let A and B be two multisets of tuples in $\mathbb{F}^n$. Define

$$\mathcal{H}_{\tau,\gamma}(X) \;=\; \prod_{x \in X} \big(h_\gamma(x) - \tau\big).$$

Then comparing $\mathcal{H}_{\tau,\gamma}(A)$ and $\mathcal{H}_{\tau,\gamma}(B)$ yields a randomized test for whether A and B are permutations of one another.Concretely:

- (Completeness) If $A = B$ (as multisets), then

$$\mathcal{H}_{\tau,\gamma}(A) = \mathcal{H}_{\tau,\gamma}(B)$$

  with probability 1 over uniform $\tau, \gamma \in \mathbb{F}$.

- (Soundness) If $A \neq B$, then

$$\Pr\left[\mathcal{H}_{\tau,\gamma}(A) = \mathcal{H}_{\tau,\gamma}(B)\right] \;\leq\; \frac{\max(|A|, |B|)}{|\mathbb{F}|}.$$

Example

Consider all operations in $\mathbb{F}_7$, set $n = 3$, $\tau = 5$, and $\gamma = 3$.
Let $A = \{(1, 2, 3), (4, 0, 6)\}, \quad B = \{(4, 0, 6), (1, 2, 3)\}$.

Example

Consider all operations in $\mathbb{F}_7$, set $n = 3$, $\tau = 5$, and $\gamma = 3$.
Let $A = \{(1, 2, 3), (4, 0, 6)\}, \quad B = \{(4, 0, 6), (1, 2, 3)\}$.

First compute the Reed–Solomon fingerprints modulo 7:

$$h_\gamma(1, 2, 3) = 1 \cdot 3^0 + 2 \cdot 3^1 + 3 \cdot 3^2 = 1 + 6 + 6 = 13 \equiv 6,$$
$$h_\gamma(4, 0, 6) = 4 \cdot 1 + 0 \cdot 3 + 6 \cdot 2 = 4 + 0 + 12 = 16 \equiv 2.$$

Example

Consider all operations in $\mathbb{F}_7$, set $n = 3$, $\tau = 5$, and $\gamma = 3$.
Let $A = \{(1, 2, 3), (4, 0, 6)\}, \quad B = \{(4, 0, 6), (1, 2, 3)\}$.

First compute the Reed–Solomon fingerprints modulo 7:

$$h_\gamma(1, 2, 3) = 1 \cdot 3^0 + 2 \cdot 3^1 + 3 \cdot 3^2 = 1 + 6 + 6 = 13 \equiv 6,$$
$$h_\gamma(4, 0, 6) = 4 \cdot 1 + 0 \cdot 3 + 6 \cdot 2 = 4 + 0 + 12 = 16 \equiv 2.$$

Now form the shifted products:

$$\mathcal{H}_{\tau,\gamma}(A) = (6 - 5)\,(2 - 5) = 1 \cdot (-3) \equiv 4,$$
$$\mathcal{H}_{\tau,\gamma}(B) = (2 - 5)\,(6 - 5) = (-3) \cdot 1 \equiv 4,$$

so the test *accepts* A vs. B (they are indeed permutations).

Example

Consider all operations in $\mathbb{F}_7$, set $n = 3$, $\tau = 5$, and $\gamma = 3$.
Let $A = \{(1, 2, 3), (4, 0, 6)\}, \quad B = \{(4, 0, 6), (1, 2, 3)\}$.

First compute the Reed–Solomon fingerprints modulo 7:

$$h_\gamma(1, 2, 3) = 1 \cdot 3^0 + 2 \cdot 3^1 + 3 \cdot 3^2 = 1 + 6 + 6 = 13 \equiv 6,$$
$$h_\gamma(4, 0, 6) = 4 \cdot 1 + 0 \cdot 3 + 6 \cdot 2 = 4 + 0 + 12 = 16 \equiv 2.$$

Now form the shifted products:

$$\mathcal{H}_{\tau,\gamma}(A) = (6 - 5)\,(2 - 5) = 1 \cdot (-3) \equiv 4,$$
$$\mathcal{H}_{\tau,\gamma}(B) = (2 - 5)\,(6 - 5) = (-3) \cdot 1 \equiv 4,$$

so the test *accepts* A vs. B (they are indeed permutations).

Now consider a non-permutation $B' = \{(1, 2, 3), (2, 1, 3)\}$.

$$h_\gamma(2, 1, 3) = 2 \cdot 1 + 1 \cdot 3 + 3 \cdot 2 = 2 + 3 + 6 = 11 \equiv 4.$$
$$\mathcal{H}_{\tau,\gamma}(B') = (6 - 5)\,(4 - 5) = 1 \cdot (-1) \equiv 6 \neq 4,$$

so the test *rejects* A vs. B'.

Offline Memory Checking

Motivation

Consider Alice, who stores two values on Bob's dedicated server at addresses 0 and 1. Initially, Bob's memory contains

$$M = \{(0, 100),\ (1, 200)\}.$$

Alice then performs the following operations in sequence:

Motivation

Consider Alice, who stores two values on Bob's dedicated server at addresses 0 and 1. Initially, Bob's memory contains

$$M = \{(0, 100),\ (1, 200)\}.$$

Alice then performs the following operations in sequence:

1. writes 150 at address 0. Bob updates his memory to $\{(0, 150), (1, 200)\}$.

Motivation

Consider Alice, who stores two values on Bob's dedicated server at addresses 0 and 1. Initially, Bob's memory contains

$$M = \{(0, 100),\ (1, 200)\}.$$

Alice then performs the following operations in sequence:

1. writes 150 at address 0. Bob updates his memory to $\{(0, 150), (1, 200)\}$.
2. read from address 0 and obtains the reply 150.

Motivation

Consider Alice, who stores two values on Bob's dedicated server at addresses 0 and 1. Initially, Bob's memory contains

$$M = \{(0, 100),\ (1, 200)\}.$$

Alice then performs the following operations in sequence:

1. writes 150 at address 0. Bob updates his memory to $\{(0, 150), (1, 200)\}$.
2. read from address 0 and obtains the reply 150.
3. reads from address 1 and (honestly) obtains 200.

Motivation

Consider Alice, who stores two values on Bob's dedicated server at addresses 0 and 1. Initially, Bob's memory contains

$$M = \{(0, 100), (1, 200)\}.$$

Alice then performs the following operations in sequence:

1. writes 150 at address 0. Bob updates his memory to $\{(0, 150), (1, 200)\}$.
2. read from address 0 and obtains the reply 150.
3. reads from address 1 and (honestly) obtains 200.

However, Bob can cheat on the very last step by returning a wrong value:

$$(1, 200) \longrightarrow (1, 300).$$

Motivation

Consider Alice, who stores two values on Bob's dedicated server at addresses 0 and 1. Initially, Bob's memory contains

$$M = \{(0, 100), (1, 200)\}.$$

Alice then performs the following operations in sequence:

1. writes 150 at address 0. Bob updates his memory to $\{(0, 150), (1, 200)\}$.
2. read from address 0 and obtains the reply 150.
3. reads from address 1 and (honestly) obtains 200.

However, Bob can cheat on the very last step by returning a wrong value:

$$(1, 200) \longrightarrow (1, 300).$$

Without keeping an auditable record of all reads, Alice cannot later prove that all replies came from correct memory contents.

Memory Model

Each **memory cell** can be described as a tuple $(\text{addr}, \text{val}, \text{counter})$.

- addr is the address of the memory cell;
- val is the value stored at that address;
- counter is a counter that is incremented each time the value at that address is written to.

Memory Model

Each **memory cell** can be described as a tuple $(\text{addr}, \text{val}, \text{counter})$.

- addr is the address of the memory cell;
- val is the value stored at that address;
- counter is a counter that is incremented each time the value at that address is written to.

The protocol utilizes four sets of tuples:

- $\mathbf{init}$ – contains the initial memory state;
- $\mathbf{write}$ – contains memory cels that represent write operations;
- $\mathbf{read}$ – contains memory cels that represent read operations;
- $\mathbf{final}$ – contains the final memory state, where all counters are set to the last value.

Memory Operations

- **Init:** load initial state; all counters = 0
- **Read:**
  - Query untrusted memory at addr $\rightarrow$ (val,counter)
  - Append (addr,val,counter) to **reads**
  - Append (addr,val,counter+1) to **writes**
- **Write:**
  - Query untrusted memory at addr $\rightarrow$ (val,counter)
  - Append (addr,val,counter) to **reads**
  - Append (addr,**newval**,counter+1) to **writes**

Consistency Check

After all reads and writes are done, the $\mathbf{final}$ set is populated with the final memory state.

Lemma

One can check the consistency of the memory operations by verifying that:

$$\mathbf{read} \cup \mathbf{final} = \mathbf{write} \cup \mathbf{init}.$$

Equivalently, via randomized permutation check:

$$\mathcal{H}_{\tau,\gamma}(\mathbf{read}) \cdot \mathcal{H}_{\tau,\gamma}(\mathbf{final}) = \mathcal{H}_{\tau,\gamma}(\mathbf{write}) \cdot \mathcal{H}_{\tau,\gamma}(\mathbf{init}).$$

Example

Suppose $N^{1/c} = 4,\ m = 3$, and the initial memory is:

$$\mathbf{init} = \{(0, 2, 0),\ (1, 5, 0),\ (2, 7, 0),\ (3, 9, 0)\},$$

while $\mathbf{read} = \varnothing$ and $\mathbf{write} = \varnothing$.

Example

Suppose $N^{1/c} = 4,\ m = 3$, and the initial memory is:

$$\mathbf{init} = \{(0, 2, 0),\ (1, 5, 0),\ (2, 7, 0),\ (3, 9, 0)\},$$

while $\mathbf{read} = \varnothing$ and $\mathbf{write} = \varnothing$.

step	operation	$\Delta\mathbf{read}_{\text{step}}$	$\Delta\mathbf{write}_{\text{step}}$
1	read(1) $\rightarrow$ $(5, 0)$	$(1, 5, 0)$	-
2	write((1,6))	-	$(1, 6, 1)$
3	read(2) $\rightarrow$ $(7, 0)$	$(2, 7, 0)$	-
4	write((2,7))	-	$(2, 7, 1)$

Example

Suppose $N^{1/c} = 4,\ m = 3$, and the initial memory is:

$$\mathbf{init} = \{(0,2,0),\ (1,5,0),\ (2,7,0),\ (3,9,0)\},$$

while $\mathbf{read} = \varnothing$ and $\mathbf{write} = \varnothing$.

step	operation	$\Delta\mathbf{read}_{\text{step}}$	$\Delta\mathbf{write}_{\text{step}}$
1	read(1) $\rightarrow$ $(5,0)$	$(1,5,0)$	-
2	write((1,6))	-	$(1,6,1)$
3	read(2) $\rightarrow$ $(7,0)$	$(2,7,0)$	-
4	write((2,7))	-	$(2,7,1)$

$$\mathbf{read} = \{(1,5,0),\ (2,7,0)\}, \quad \mathbf{write} = \{(1,6,1),\ (2,7,1)\}$$

$$\mathbf{final} = \{(0,2,0),\ (1,6,1),\ (2,7,1),\ (3,9,0)\}$$

Example

Suppose $N^{1/c} = 4,\ m = 3$, and the initial memory is:

$$\mathbf{init} = \{(0,2,0),\ (1,5,0),\ (2,7,0),\ (3,9,0)\},$$

while $\mathbf{read} = \varnothing$ and $\mathbf{write} = \varnothing$.

step	operation	$\Delta\mathbf{read}_{\text{step}}$	$\Delta\mathbf{write}_{\text{step}}$
1	read(1) $\rightarrow$ $(5,0)$	$(1,5,0)$	-
2	write((1,6))	-	$(1,6,1)$
3	read(2) $\rightarrow$ $(7,0)$	$(2,7,0)$	-
4	write((2,7))	-	$(2,7,1)$

$$\mathbf{read} = \{(1,5,0),\ (2,7,0)\}, \quad \mathbf{write} = \{(1,6,1),\ (2,7,1)\}$$
$$\mathbf{final} = \{(0,2,0),\ (1,6,1),\ (2,7,1),\ (3,9,0)\}$$

One can clearly see that $\mathbf{read} \cup \mathbf{final} = \mathbf{write} \cup \mathbf{init}$.

$$\{(1,5,0),(2,7,0)\} \cup \{(0,2,0),(1,6,1),(2,7,1),(3,9,0)\} =$$
$$= \{(1,6,1),(2,7,1)\} \cup \{(0,2,0),(1,5,0),(2,7,0),(3,9,0)\}$$

Example

Suppose $N^{1/c} = 4,\ m = 3$, and the initial memory is:

$$\mathbf{init} = \{(0, 2, 0),\ (1, 5, 0),\ (2, 7, 0),\ (3, 9, 0)\},$$

while $\mathbf{read} = \varnothing$ and $\mathbf{write} = \varnothing$.

Example

Suppose $N^{1/c} = 4,\ m = 3$, and the initial memory is:

$$\mathbf{init} = \{(0, 2, 0),\ (1, 5, 0),\ (2, 7, 0),\ (3, 9, 0)\},$$

while $\mathbf{read} = \varnothing$ and $\mathbf{write} = \varnothing$.

step	**operation**	$\Delta\mathbf{read}_{\text{step}}$	$\Delta\mathbf{write}_{\text{step}}$
1	read(1) $\rightarrow\ (a, 0)$	$(1, a, 0)$	-
2	write((1,6))	-	$(1, 6, 1)$

Example

Suppose $N^{1/c} = 4,\ m = 3$, and the initial memory is:

$$\mathbf{init} = \{(0,2,0),\ (1,5,0),\ (2,7,0),\ (3,9,0)\},$$

while $\mathbf{read} = \varnothing$ and $\mathbf{write} = \varnothing$.

step	operation	$\Delta\mathbf{read}_{\text{step}}$	$\Delta\mathbf{write}_{\text{step}}$
1	read(1) $\rightarrow\ (a,0)$	$(1,a,0)$	-
2	write((1,6))	-	$(1,6,1)$

$$\mathbf{read} = \{(1,a,0)\}, \quad \mathbf{write} = \{(1,6,1)\},$$
$$\mathbf{final} = \{(0,2,0),\ (1,6,1),\ (2,7,1),\ (3,9,0)\}$$

Example

Suppose $N^{1/c} = 4,\ m = 3$, and the initial memory is:

$$\mathbf{init} = \{(0,2,0),\ (1,5,0),\ (2,7,0),\ (3,9,0)\},$$

while $\mathbf{read} = \varnothing$ and $\mathbf{write} = \varnothing$.

step	operation	$\Delta\mathbf{read}_{\text{step}}$	$\Delta\mathbf{write}_{\text{step}}$
1	read(1) $\rightarrow\ (a,0)$	$(1,a,0)$	-
2	write((1,6))	-	$(1,6,1)$

$$\mathbf{read} = \{(1,a,0)\}, \quad \mathbf{write} = \{(1,6,1)\},$$
$$\mathbf{final} = \{(0,2,0),\ (1,6,1),\ (2,7,1),\ (3,9,0)\}$$

The verifier checks $\mathbf{read} \cup \mathbf{final} = \mathbf{write} \cup \mathbf{init}$:

$$\{(1,a,0)\} \cup \{(0,2,0),(1,6,1),(2,7,0),(3,9,0)\} \neq$$
$$\neq \{(1,6,1)\} \cup \{(0,2,0),(1,5,0),(2,7,0),(3,9,0)\}$$

and *rejects*.

Questions?